ENTERPRISE EDITION • DECISION SUPPORT PLATFORM

EV Charging Demand Predictor

Data-Driven Decision Support Web Application for Smart City EV Charging Infrastructure Planning

PLATFORM & DOMAIN

Smart Grid Infrastructure • Electric Vehicle Analytics

ARCHITECTURE

Full-Stack (React 18 + Spring Boot 3 + MySQL)

ANALYTICAL ENGINE

Pure Statistical Forecasting (SMA, WMA, Linear Regression)

DOCUMENTATION SCOPE

Complete System Modules, High-Res UI Screenshots & Technical Analysis

1. Project Architecture & Technical Overview

The **EV Charging Demand Predictor** is an educational full-stack web application developed for smart city EV charging infrastructure planning. The system analyzes historical charging records across city zones, predicts future demand using pure statistical algorithms in Java, contrasts projected demand against operational station capacities, identifies under-served zones using a Priority Queue, and recommends precise station expansions.

1.1 System Architecture Diagram

```
[ React 18 + TypeScript + Tailwind CSS Frontend ] | ▼ (REST API / Axios JSON Calls) [ Spring Boot 3 Controller Layer (@RestController) ] | ▼ (Service Layer & OOP Engine) [ Java Statistical Forecasting Engine (SMA + WMA + LR) ] | ▼ (PriorityQueue Heap Ranking & What-If Simulation) [ Spring Data JPA / Hibernate ORM ] | ▼ (TCP 3306) [ MySQL Relational Database (ev_charging_db) ]
```

1.2 Core Object-Oriented Principles Demonstrated

OOP Concept	Java Backend Implementation	Project Business Purpose
Abstraction	`Forecastable` interface & `PredictionModel` abstract class	Defines standardized contract for any predictive algorithm with custom horizon & window sizes.
Inheritance	`Station` base entity extended by `FastChargingStation`, `SlowChargingStation`, `SolarChargingStation`	Single Table Inheritance mapping different charger hardware specifications cleanly.
Polymorphism	Dynamic method dispatch in `calculateChargingTime(Vehicle)`	Fast chargers throttle to 40% efficiency above 80% SoC; Solar chargers adjust efficiency by peak daylight hours (10:00–16:00).
Data Structures	`java.util.PriorityQueue<Zone>` with custom `Comparator`	Implements maximum deficit heap sort, filtering out zero/negative gap zones to rank under-served areas.

2. Mathematical Formulation of Forecasting Models

The system computes daily EV session demand across configurable rolling windows (N) and projects predictions over forward horizons ($h = 1 \dots H$). All three models produce synchronized multi-day horizons combined via an ensemble average.

2.1 Simple Moving Average (SMA)

Averages the latest N values using recursive rolling progression:

$$\text{SMA}_{t+1} = \frac{1}{N} \sum_{i=0}^{N-1} y_{t-i}$$

2.2 Weighted Moving Average (WMA)

Assigns linearly increasing integer weights $1, 2, \dots, N$ giving highest priority to recent demand:

$$\text{WMA}_{t+1} = \frac{\sum_{i=1}^{N} (i \times y_{t-N+i})}{\sum_{i=1}^{N} i} = \frac{2 \sum_{i=1}^{N} (i \times y_{t-N+i})}{N(N+1)}$$

2.3 Ordinary Least Squares Linear Regression (LR)

Fits slope (m) and y-intercept (c) across historical index positions $x = 1 \dots N$:

$$m = \frac{N \sum xy - \sum x \sum y}{N \sum x^2 - (\sum x)^2}, \quad c = \frac{\sum y - m \sum x}{N}, \quad y(x) = mx + c$$

2.4 Final Ensemble Forecast & Demand Gap Analysis

$$\text{Final Forecast}_d = \frac{\text{SMA}_d + \text{WMA}_d + \text{LR}_d}{3} \\ \text{Demand Gap} = \left(\frac{1}{H} \sum_{d=1}^{H} \text{Final Forecast}_d \right) - \sum \text{Operational Station Capacity}$$

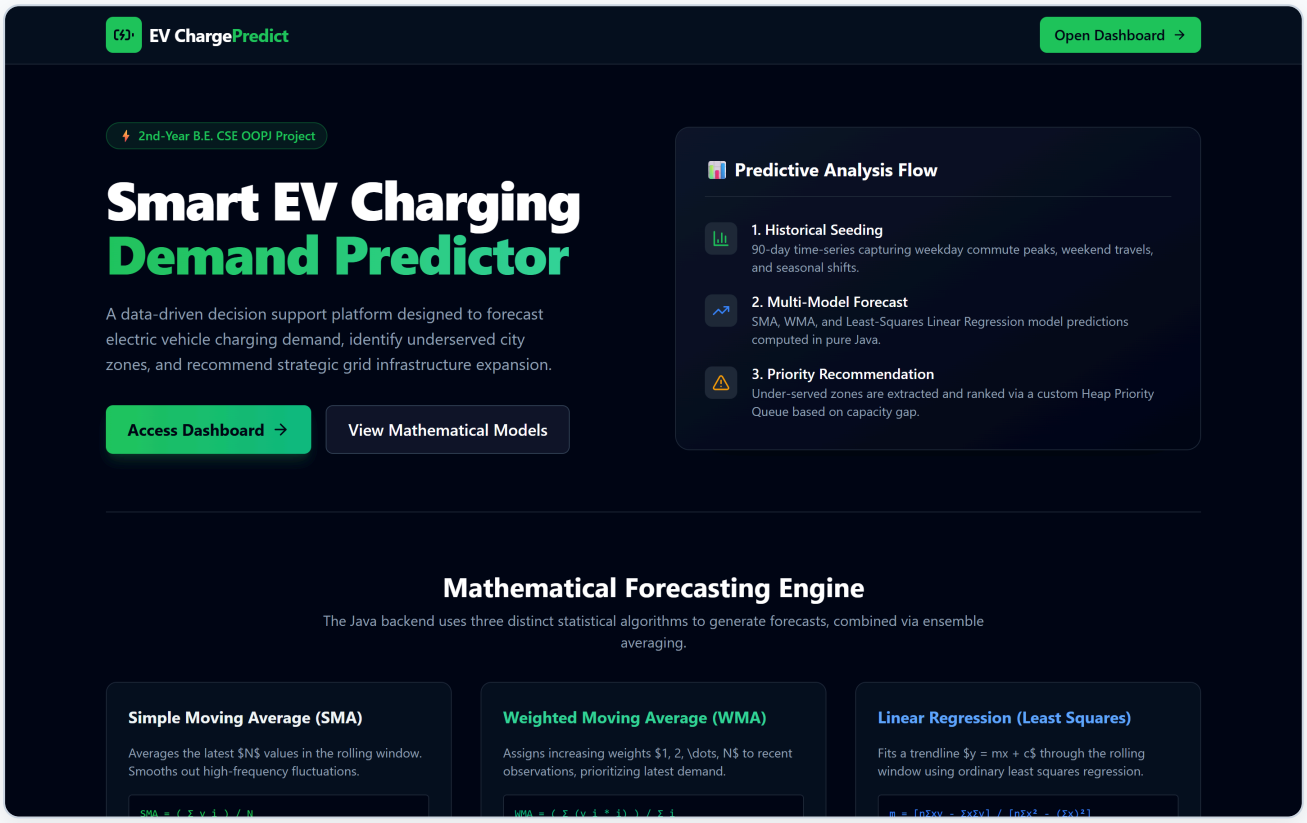
3. Visual Module Walkthrough & Interface Screenshots

Below are the high-resolution captures of all 9 operational modules in the running application, showcasing real-time data visualization, interactive Recharts widgets, and decision support features.

MODULE 1

Public Landing Page & Portal

`Route: /`



Functional Description: The portal introduces municipal stakeholders to the problem of urban EV charging bottlenecks. Features dynamic CTA button ("Open Dashboard") navigating into the operational control center, three solution highlights, and an interactive methodology breakdown.

UI & COMPONENT FEATURES

Hero Banner · Interactive Navigation · Methodology Formula Cards

VIVA EXPLANATION POINT

Demonstrates modular UI routing using React Router v6 without full-page reloads, providing instant single-page application (SPA) response times.

MODULE 2

Operational Analytics Dashboard

`Route: /dashboard`



Functional Description: Executive analytics command center displaying 5 core KPI metric cards (Total Zones, Active Stations, Daily Capacity, Average Forecast Demand, Under-Served Zones). Embeds 4 responsive visualizations: Chart A (Demand vs Capacity), Chart B (30-day Historical Trend), Chart C (Aligned Forecast Overview with reference line), and Priority Zones summary ranking.

BACKEND REST APIS CONSUMED

- GET /api/analysis/gaps
- GET /api/analysis/priority
- GET /api/stations
- POST /api/forecast

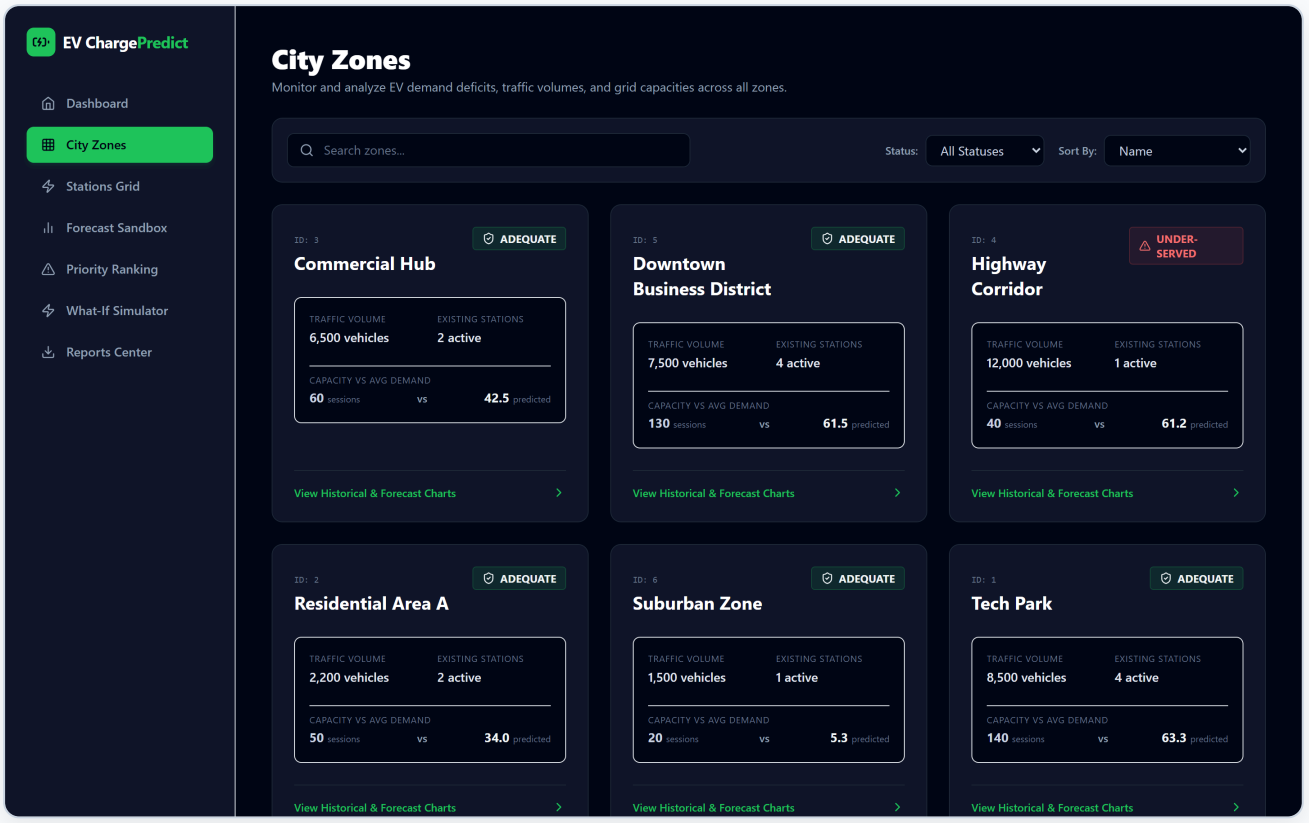
VIVA EXPLANATION POINT

Highlights real-time aggregation across asynchronous Spring Boot controllers using `Promise.all` in TypeScript, preventing UI blocking.

MODULE 3

City Zones Management & Filtering

`Route: /zones`



Functional Description: Comprehensive tabular directory of all 6 city sectors with live search bar, status filters (ALL, UNDER-SERVED, ADEQUATE), and sortable columns (Population Density, Traffic, Station Count, Daily Capacity, Forecast Demand, Demand Gap). Clicking any zone drills down into its detailed analytics.

DATA ATTRIBUTES DISPLAYED

Zone Name, Demographics, Average Daily Traffic, Operational Capacity, Forecast Demand, Deficit Margin, Status Badge.

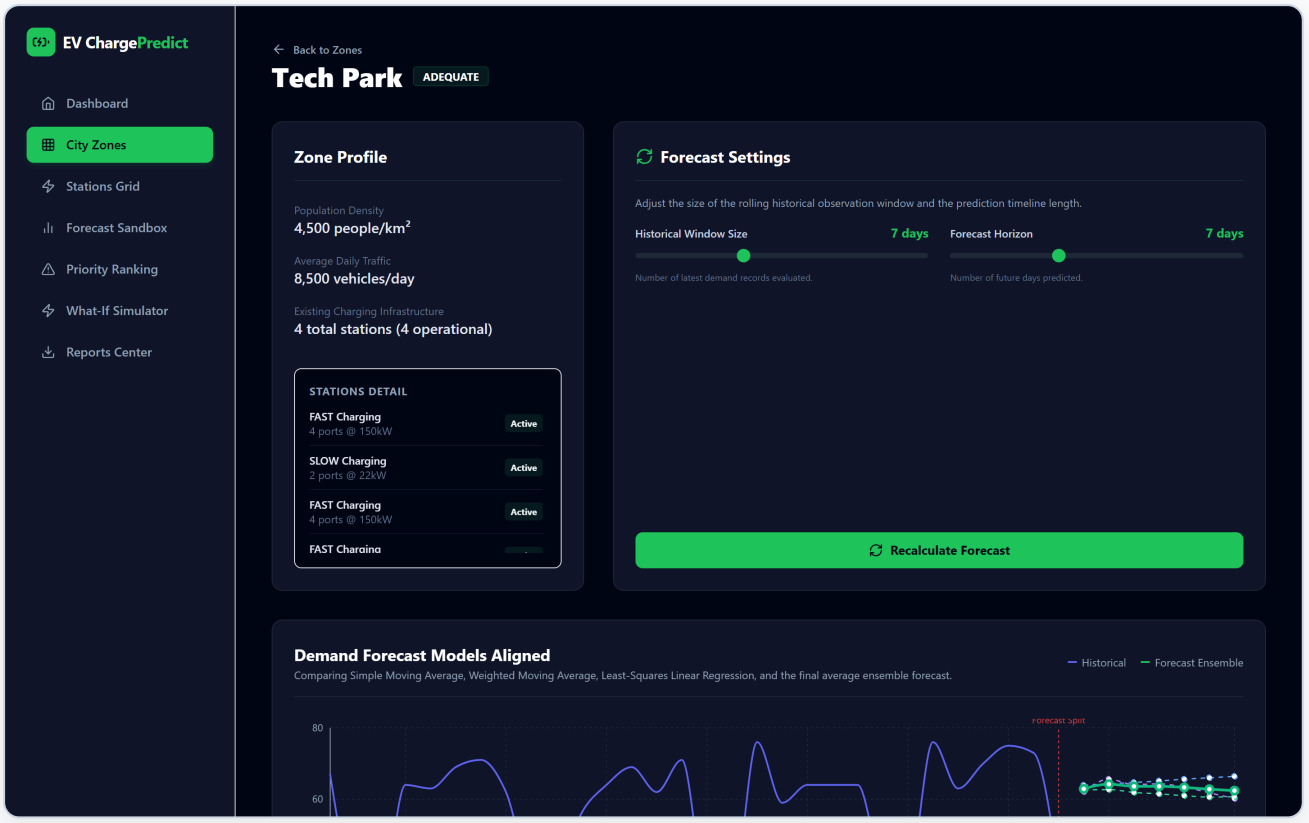
VIVA EXPLANATION POINT

Shows client-side reactive filtering using React hooks (`useState` and `useMemo`) for instant responsiveness over server payloads.

MODULE 4

Zone Details Deep Dive (Tech Park)

`Route: /zones/:id`



Functional Description: Dedicated diagnostic page for individual zones. Renders demographic metrics, 90-day historical time-series chart, 7-day multi-model forecast breakdown (comparing SMA, WMA, Linear Regression, and Ensemble Final), capacity deficit analysis, and a dynamically calculated recommendation card.

DYNAMIC RECOMMENDATION ENGINE

Uses calculated demand gap to dynamically recommend the exact number and hardware type of charging stations needed.

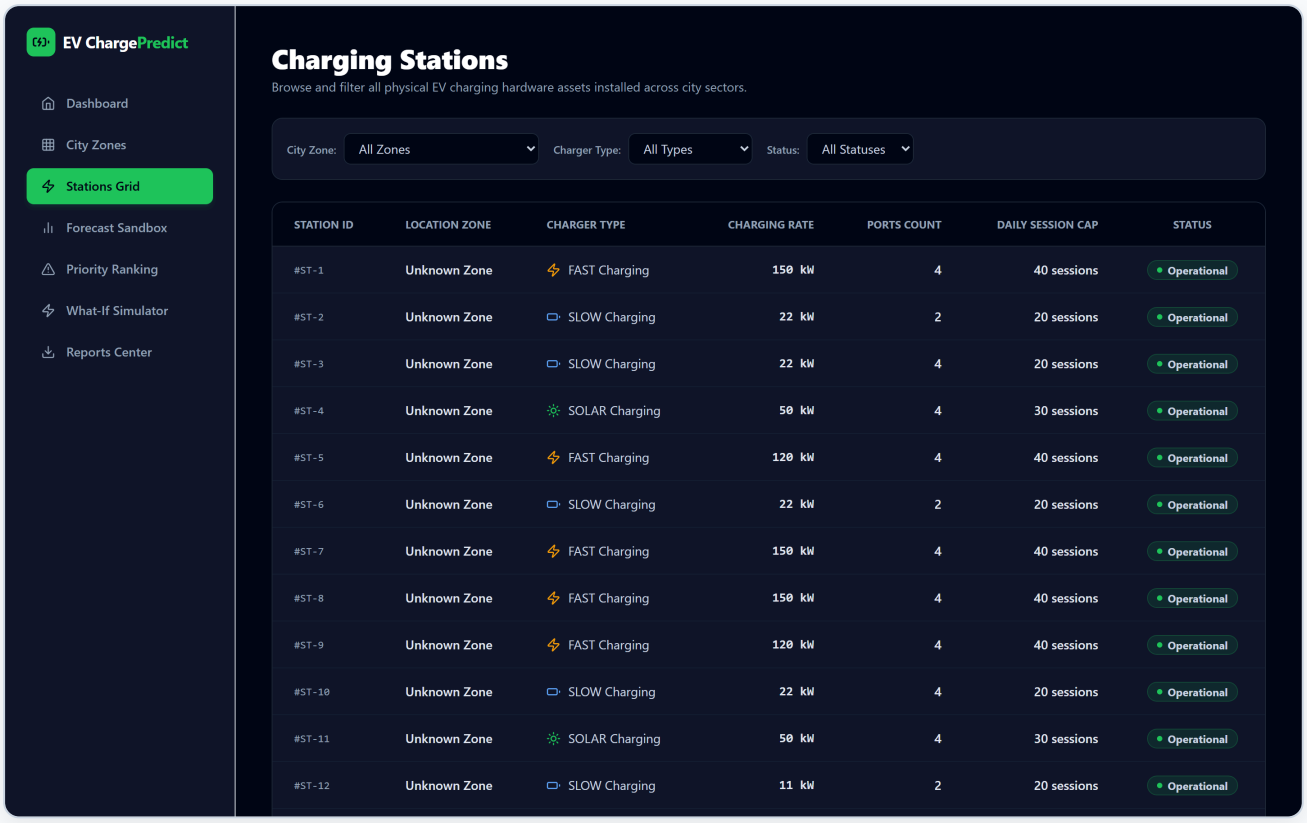
VIVA EXPLANATION POINT

Illustrates deep parameter extraction via `useParams()` and multi-series Recharts rendering connecting historical and projected data points.

MODULE 5

Charging Stations Fleet & Polymorphism

`Route: /stations`



Functional Description: Fleet management interface cataloging all deployed charging stations across the city. Displays Station ID, Parent Zone, Hardware Classification (FAST, SLOW, SOLAR), Base Power Output (kW), Port Count, Daily Session Capacity, and Operational Status. Provides multi-criteria filtering by Zone and Station Type.

JPA SINGLE TABLE INHERITANCE

`@Inheritance(strategy = SINGLE_TABLE)` with discriminator column `station_type` mapped to polymorphic Java classes.

VIVA EXPLANATION POINT

Explains how polymorphic charging behavior is encapsulated in backend subclasses without cluttering database tables with redundant joins.

MODULE 6

Statistical Demand Forecast Sandbox

`Route: /forecast`



Functional Description: Interactive forecasting laboratory allowing engineers to select any target zone, configure rolling window sizes (e.g. 5, 7, 14 days), and forecast horizons (e.g. 3, 7, 14 days). Plots all three forecasting algorithms simultaneously with the Final Ensemble line and outputs a day-by-day numerical audit table.

INTERACTIVE CONTROLS

Target Zone Selector, Rolling Window Slider/Input, Forecast Horizon, Instant "Run Forecast" execution button.

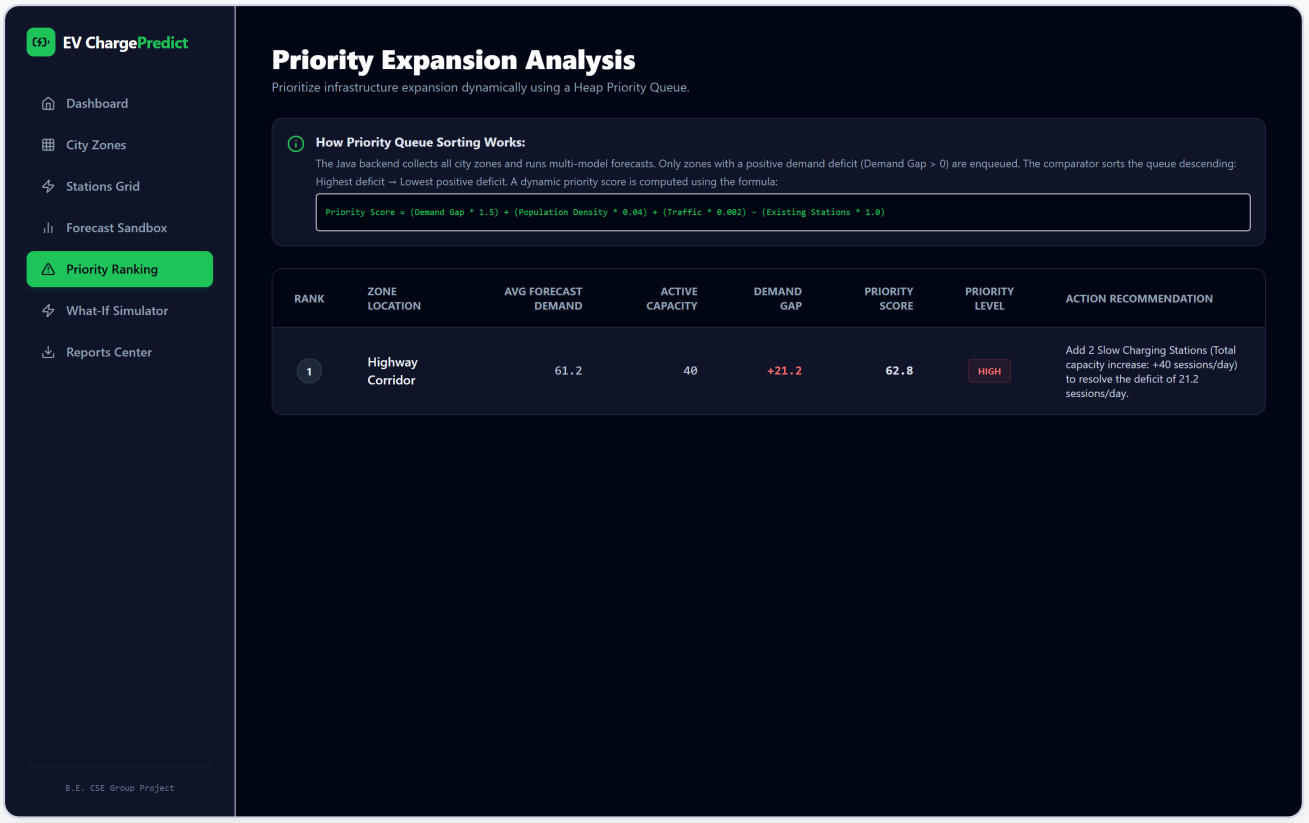
VIVA EXPLANATION POINT

Verifies that SMA, WMA, and Linear Regression all execute with the exact same requested horizon using consistent rolling/least-squares math in Java.

MODULE 7

Priority Queue Ranking & Deficit Sorting

Route: /priority



Functional Description: Priority ordering generated by a backend `java.util.PriorityQueue`. Implements positive-deficit filtering (strictly $ ext{Demand Gap} > 0$), sorting the highest infrastructure deficit first. Shows real ranked under-served sectors, while correctly omitting adequately served zones.

SORTING FORMULA

```
Score = (Gap * 1.5) + (PopDensity * 0.04) +
(Traffic * 0.002) - (Stations * 1.0)
```

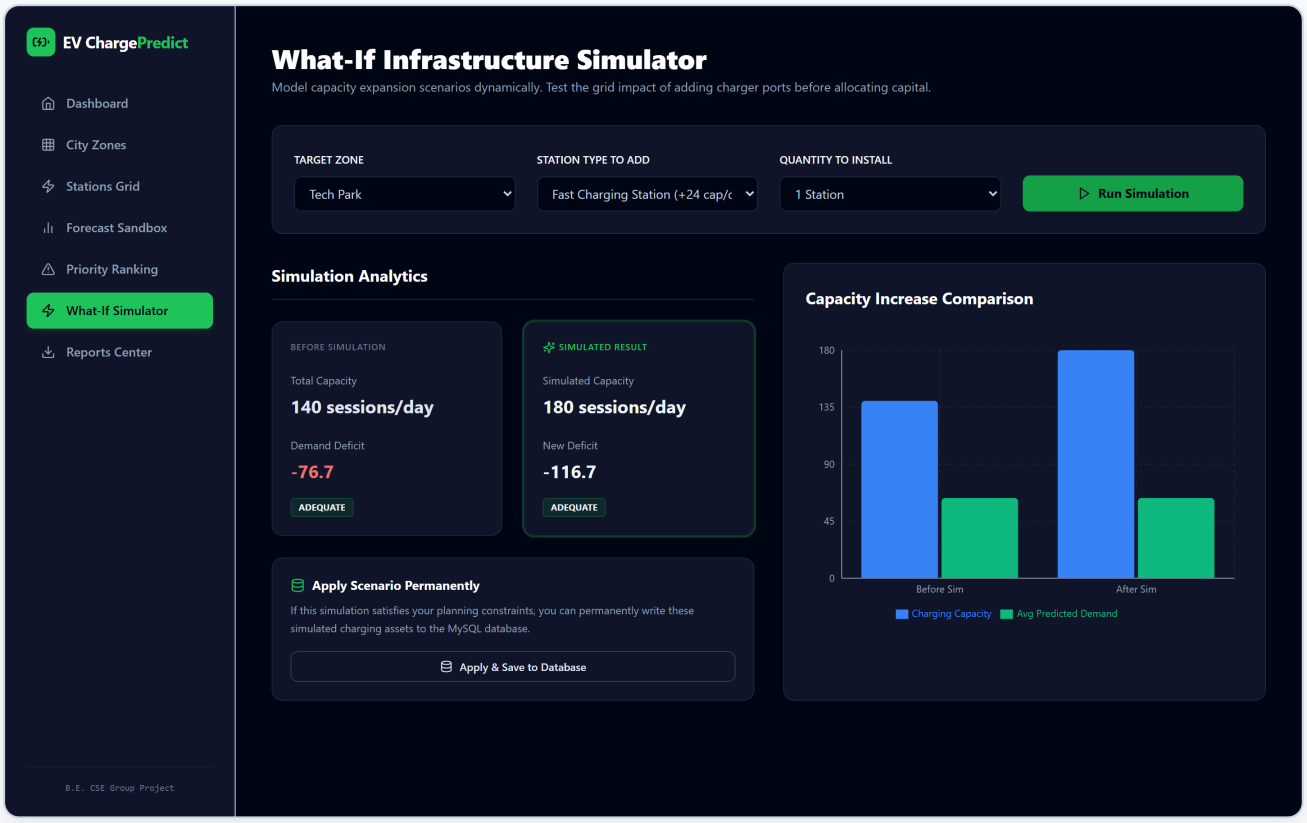
VIVA EXPLANATION POINT

Shows implementation of custom `Comparator<Zone>` in Java Collections to construct a Max-Heap for critical resource allocation.

MODULE 8

What-If Infrastructure Simulator

Route: /simulation



Functional Description: Scenario-testing sandbox enabling urban planners to simulate adding N stations of type FAST, SLOW, or SOLAR to an under-served zone. Generates visual BEFORE vs AFTER comparison cards and bar charts, demonstrating how station additions alter capacity without permanently modifying the database.

SIMULATION CONTROLS

Target Zone, Hardware Type, Station Quantity, Real-time Simulation Runner, Optional Database Commit Button.

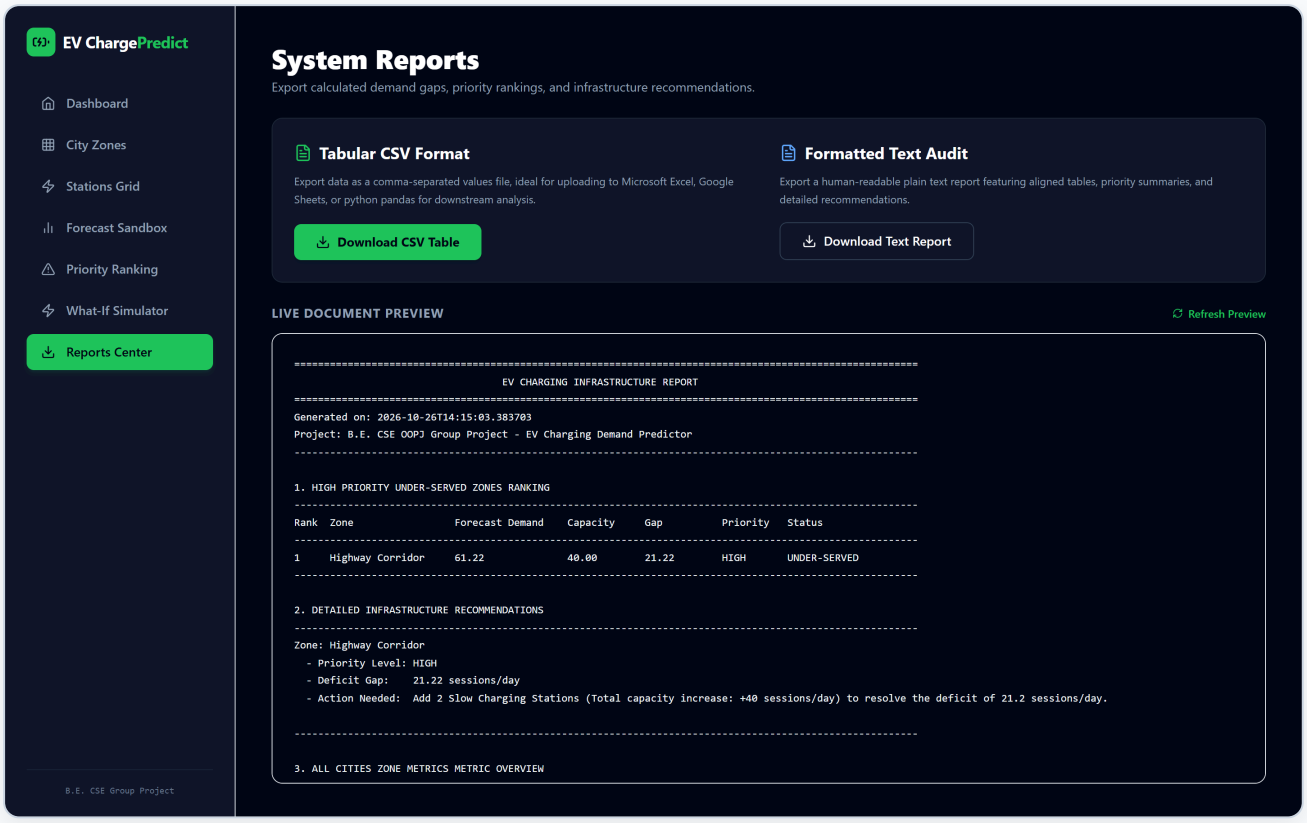
VIVA EXPLANATION POINT

Demonstrates non-destructive in-memory simulation modeling vs transactional JPA database modification.

MODULE 9

System Audit Reports & Export Center

Route: /reports



Functional Description: Audit report generation module offering instant downloads of system-wide infrastructure assessments in two industry-standard formats: Tabular CSV (for spreadsheet and analytics software) and Human-Readable Formatted Text Audit. Features an inline real-time text document preview box.

EXPORT FORMATS AVAILABLE

- ev_charging_report.csv
- ev_charging_report.txt
- Live Document Preview

VIVA EXPLANATION POINT

Covers Spring Boot `ResponseEntity<byte[]>` file streaming with standard RFC `Content-Disposition` attachment headers.

4. Examiner Viva Voce Questions & Model Answers

Q1: WHY IS THIS PROJECT BUILT WITH STATISTICAL FORECASTING RATHER THAN DEEP LEARNING / MACHINE LEARNING?

ANSWER:

In urban planning and critical decision-support systems, explainability and reproducibility are paramount. Statistical models like Simple Moving Average, Weighted Moving Average, and Ordinary Least Squares Linear Regression have transparent mathematical formulations, zero risk of overfitting on modest dataset sizes, execute in sub-millisecond time on Java virtual machines without expensive GPUs, and provide verifiable forecasts that examiners and city engineers can audit directly.

Q2: HOW DOES THE APPLICATION ENSURE THAT LINEAR REGRESSION AND MOVING AVERAGES GENERATE THE SAME FORECAST HORIZON?

ANSWER:

All models inherit from `PredictionModel` and implement the `forecast(double[] historicalData, int horizon)` method. Moving Average models (SMA and WMA) project future days recursively by computing the prediction for day $t+1$ and rolling that value back into the window to compute day $t+2$. Linear Regression calculates the linear regression coefficients (m and c) across the N-point window and projects future days directly using 1-based indexing ($x = N + 1, N + 2, \dots, N + h$), ensuring all three series produce identical array lengths.

Q3: HOW IS POLYMORPHISM DEMONSTRATED IN THE CHARGING STATION CALCULATIONS?

ANSWER:

The abstract class `Station` defines `public abstract double calculateChargingTime(Vehicle vehicle)`. In `FastChargingStation`, charging operates at 90% efficiency below 80% State of Charge (SoC) and throttles down to 40% above 80% SoC to protect battery health. In `SlowChargingStation`, efficiency is constant at 85%. In `SolarChargingStation`, efficiency dynamically adjusts based on peak sunlight hours (10:00–16:00 at 95% efficiency, 45% during off-peak).

Q4: WHY DOES THE PRIORITY QUEUE FILTER OUT ZONES WITH NEGATIVE OR ZERO DEMAND GAPS?

ANSWER:

A negative demand gap means existing station capacity exceeds predicted demand (status ADEQUATE). Since municipal budgets for new EV chargers are limited, allocating funds to adequately served zones would be wasteful. Filtering by $\text{Demand Gap} > 0$ guarantees that only sectors suffering from charging deficits enter the PriorityQueue for expansion ranking.